

designing quora website through html and css

Great practice idea, Aamir 🧑💻

Let's design a **Quora-style website UI** using **pure HTML & CSS** (no JS) — clean, professional, and beginner-friendly.

🧩 Project Structure

Create two files:

quora.html
style.css

📄 Copy code

📄 quora.html

html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Quora Clone</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <div class="logo">Quora</div>
  <nav>
    <a href="#">Home</a>
    <a href="#">Answer</a>
    <a href="#">Spaces</a>
```

```
<a href="#">Notifications</a>
<a href="#">Profile</a>
</nav>
</header>

<main>

<div class="left">
  <h3>Topics</h3>
  <p>Technology</p>
  <p>Programming</p>
  <p>Science</p>
  <p>Education</p>
</div>

<div class="center">
  <div class="question-box">
    <h4>What is HTML and CSS?</h4>
    <p>HTML is the structure of a webpage and CSS is used for styling it.</p>
    <button>Answer</button>
  </div>

  <div class="question-box">
    <h4>Why is JavaScript important?</h4>
    <p>JavaScript makes websites interactive and dynamic.</p>
    <button>Answer</button>
  </div>
</div>

<div class="right">
  <h3>Trending</h3>
  <p>Web Development</p>
  <p>AI & ML</p>
  <p>Data Science</p>
</div>

</main>
```

```
</body>
```

```
</html>
```

style.css

CSS

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
  font-family: Arial, Helvetica, sans-serif;  
}
```

```
body {  
  background: #f1f2f2;  
}
```

```
/* Header */
```

```
header {  
  background: white;  
  padding: 15px 40px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  border-bottom: 1px solid #ddd;  
}
```

```
.logo {  
  color: #b92b27;  
  font-size: 26px;  
  font-weight: bold;  
}
```

```
nav a {  
  margin-left: 20px;  
  text-decoration: none;
```

```
        color: #333;
        font-weight: 600;
    }

    /* Layout */

    main {
        display: flex;
        padding: 20px 40px;
        gap: 20px;
    }

    .left, .right {
        width: 20%;
        background: white;
        padding: 15px;
        border-radius: 6px;
    }

    .center {
        width: 60%;
    }

    /* Question Box */

    .question-box {
        background: white;
        padding: 15px;
        border-radius: 6px;
        margin-bottom: 15px;
    }

    .question-box h4 {
        margin-bottom: 8px;
    }

    .question-box p {
        color: #555;
        margin-bottom: 10px;
    }
}
```

```
.question-box button {  
  padding: 6px 12px;  
  border: none;  
  background: #b92b27;  
  color: white;  
  border-radius: 4px;  
  cursor: pointer;  
}
```



How It Looks

- ✓ Top navbar
- ✓ 3-column layout (like Quora)
- ✓ Question cards
- ✓ Clean modern UI



Concepts You Practice Here

- Flexbox layout
- Page structuring
- Real-world UI design
- Box model
- Positioning and spacing

If you want, next I can:

- ◆ Add **login page**
- ◆ Make it **responsive**
- ◆ Add **hover effects & animations**
- ◆ Convert it into **React project**

Just tell me 



universal selector

Great — **Universal Selector (*)** is a very important CSS concept 👍

Let's understand it **deeply but simply**.

 Copy code

What is Universal Selector?

The **universal selector** is written as:

CSS

```
*
```

 Copy code

It selects **every element on the webpage**.

Why we use it?

Mostly used to:

- Remove default margin & padding
- Apply common styles
- Create a clean layout base

Basic Example

CSS

```
* {  
  margin: 0;  
  padding: 0;  
}
```

This removes default spacing from:

 Copy code

CSS

`body, h1, p, div, span, ul, li` – everything

 Copy code

Real Use Case (Layout Reset)

CSS

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

What each line means:

Property

Meaning

 Copy code

`margin: 0`

removes outside spacing

`padding: 0`

removes inside spacing

`box-sizing: border-box`

keeps width/height stable

This is used in **almost every professional project**.

Apply Common Styles

CSS

```
* {  
  font-family: Arial;  
  color: #333;  
}
```

All text will follow this unless overridden.

 Copy code

Specificity Rule

Universal selector has the **lowest priority**.

CSS

```
* { color: red; }  
p { color: blue; }
```

Paragraph will be **blue**, not red.

 Copy code

What NOT to do

Avoid heavy styles:

CSS

```
* {  
  background: red;   
}
```

It applies to everything → messy & slow.

 Copy code

Interview Question

Q: Why do developers use * selector at the start of CSS?

Answer:

To reset default browser spacing and create consistent layout behavior.

Small Demo

CSS

```
* {  
  margin: 0;  
  padding: 0;  
}
```

```
html
```

```
<h1>Hello</h1>
```

```
<p>World</p>
```

Without * → space around elements

 Copy code

With * → no space

If you want, next I can explain:

- **Element selector**
- **Class vs ID selector**
- **Attribute selectors**
- **CSS Specificity system**

You're learning real frontend now, Aamir  

     

element selector

Perfect 

Element Selector is one of the most basic and powerful CSS selectors.

Let's break it down clearly 

What is Element Selector?

An **element selector** selects HTML elements **by their tag name**.

Syntax:

```
CSS
```

```
tagname {  
  property: value;
```

```
}
```



Example:

[Copy code](#)

CSS

```
p {  
  color: blue;  
}
```

This styles **all** `<p>` **elements** on the page.

[Copy code](#)

Common Element Selectors

CSS

```
h1 { }  
p { }  
div { }  
span { }  
img { }  
a { }  
button { }
```



How it Works

[Copy code](#)

HTML:

html

```
<p>Hello</p>  
<p>World</p>
```

CSS:

[Copy code](#)

CSS

```
p {  
  color: red;  
  font-size: 18px;  
}
```

 Copy code

Both paragraphs become red & 18px.



Multiple Elements at Once

CSS

```
h1, h2, h3 {  
  color: darkgreen;  
}
```

 Copy code

Element Selector + Descendant

CSS

```
div p {  
  color: purple;  
}
```

This selects only `<p>` inside `<div>` .

 Copy code

Specificity Level

Selector	Priority
Universal *	Lowest
Element p	Low
Class .box	Medium
ID #main	High

Selector	Priority
Inline style	Highest

So this wins:

CSS

```
p { color: blue; }  
.text { color: red; }
```

If `<p class="text">` → color will be red.

 Copy code

Common Mistake

CSS

```
#p { } ❌ (this is ID selector, not element)  
.p { } ❌ (this is class selector)  
p { } ✅ element selector
```

 Copy code

Real Use Case

CSS

```
body {  
  background: #f5f5f5;  
}  
  
a {  
  text-decoration: none;  
}  
  
img {  
  max-width: 100%;  
}
```



Mini Practice

Question:

Make all headings blue and all paragraphs gray.

Copy code

CSS

```
h1, h2, h3 {  
  color: blue;  
}
```

```
p {  
  color: gray;  
}
```

If you want, next we go to:

Copy code

► Class Selector vs ID Selector

(super important for interviews & real projects)



id selector

Great 👍

Now you're reaching the **most important CSS selector** for layout & JS control:



ID Selector

Let's understand it properly 🙋



What is ID Selector?

An **ID selector** selects **one unique element** using its `id` attribute.



Syntax:

CSS

```
#idName {  
    property: value;  
}
```

 Copy code

Example

HTML:

html

```
<h1 id="title">Welcome</h1>  
<p>This is a paragraph</p>
```

CSS:

 Copy code

CSS

```
#title {  
    color: red;  
    font-size: 30px;  
}
```

Only the `<h1>` becomes red & 30px.

 Copy code

Key Rules

Rule	Explanation
ID must be unique	Only one element should have that ID
One element → one ID	You should not reuse the same ID
ID has high priority	Higher than class & element



Specificity Order

CSS

```
p { color: blue; }           /* Low */
.text { color: green; }     /* Medium */
#title { color: red; }      /* High */
```

Result: red

Copy code



Multiple Properties Example

CSS

```
#loginBtn {
  background: #b92b27;
  color: white;
  padding: 10px 15px;
  border-radius: 5px;
}
```



Common Mistakes

Copy code

html

```
<p id="box"></p>
<div id="box"></div> ❌ wrong (duplicate ID)
```

Copy code



Real Use Case

Used when:

- ✓ Targeting main containers
- ✓ JS manipulation
- ✓ Important unique sections

html

```
<div id="header"></div>
<div id="sidebar"></div>
<div id="footer"></div>
```

[Copy code](#)

ID vs Class

Feature	ID	Class
Symbol	#	.
Unique	Yes	No
Priority	High	Medium
Reusable	✗	✓

Mini Practice

Style the login button:

html

```
<button id="login">Login</button>
```

css

```
#login {
  background: green;
  color: white;
  padding: 10px;
}
```

[Copy code](#)

Next concept you **must** learn:

► Class Selector

(after that: **attribute selector & pseudo-classes**)



class selector

Excellent — now you're at the **most used selector in real projects** 🏆

Class Selector

What is Class Selector?

A **class selector** styles **one or many elements** using the `class` attribute.

Syntax:

CSS

```
.className {  
  property: value;  
}
```

 Copy code

Basic Example

HTML:

html

```
<p class="text">Hello</p>  
<p class="text">World</p>
```

CSS:

 Copy code

CSS

```
.text {  
  color: blue;  
  font-size: 18px;  
}
```

 Copy code

Both paragraphs become blue & 18px.

 Key Advantages

Feature	Class
Reusable	✓ Yes
Multiple per element	✓ Yes
Good for layouts	✓ Yes
Priority	Medium

 One Element → Multiple Classes

html

```
<div class="card shadow large"></div>
```

 Copy code

css

```
.card { background: white; }  
.shadow { box-shadow: 0 0 5px gray; }  
.large { width: 300px; }
```

 Copy code

 Priority Comparison

css

```
p { color: blue; }  
.text { color: green; }  
#title { color: red; }
```

 Copy code

For:

html

```
<p class="text" id="title">Hello</p>
```

 Copy code

Result → Red (ID wins)



Real Use Case

CSS

```
.container { width: 80%; margin: auto; }  
.btn { padding: 8px 15px; border-radius: 5px; }  
.btn-primary { background: blue; color: white; }
```

 Copy code

Common Mistakes

CSS

```
#text { } ❌ ID selector  
text { } ❌ element selector  
.text { } ✅ class selector
```

 Copy code

Mini Practice

Make 3 boxes using one class:

html

```
<div class="box"></div>  
<div class="box"></div>
```

```
<div class="box"></div>
```

CSS

```
.box {  
  width: 100px;  
  height: 100px;  
  background: orange;  
}
```



Class vs ID

Copy code

Feature	Class	ID
Symbol	.	#
Reuse	✓	✗
Specificity	Medium	High
JS targeting	Often	Often

Next powerful concepts you should learn:

- Attribute Selector
- Pseudo-classes (:hover , :focus)
- Pseudo-elements (::before , ::after)



Descendant selector

Excellent — **Descendant Selector** is a core concept for real layouts & clean CSS 🔥
Let's understand it step-by-step.

🌳 What is Descendant Selector?

It selects an element **inside another element**, no matter how deep it is.

🔧 Syntax:

CSS

```
parent child {  
  property: value;  
}
```

It means:

 Copy code

Select all **child elements inside parent**

🧱 Basic Example

HTML:

html

```
<div>  
  <p>Hello</p>  
</div>  
  
<p>World</p>
```

CSS:

CSS

```
div p {  
  color: red;  
}
```

Result:

[Copy code](#)

- Hello → **Red**
- World → unchanged

Because only `<p>` inside `<div>` is selected.

Multi-level Descendant

CSS

```
section div p {  
    color: blue;  
}
```

Selects `<p>` inside `<div>` inside `<section>`.

[Copy code](#)

Why Use It?

- ✓ Cleaner CSS
 - ✓ Avoid unnecessary classes
 - ✓ Target specific sections
 - ✓ Essential for layout control
-

Real Website Example

CSS

```
header nav a {  
    text-decoration: none;  
    color: black;  
}
```

Targets only links inside navbar.

[Copy code](#)

Difference: Descendant vs Child

Selector	Meaning
<code>div p</code>	Any <code><p></code> inside <code><div></code>
<code>div > p</code>	Only direct child <code><p></code>



Compare Example

html

```
<div>
  <p>One</p>
  <section>
    <p>Two</p>
  </section>
</div>
```

css

```
div p { color: red; }    /* One & Two */
div > p { color: blue; } /* Only One */
```

Copy code



Specificity

`div p` = two element selectors → stronger than just `p`.



Mini Practice

Style only menu links:

html

```
<nav>
  <a>Home</a>
  <a>About</a>
</nav>
```

```
<a>Login</a>
```

CSS

```
nav a {  
  color: green;  
}
```

 Copy code

Interview Question

Q: What is a descendant selector?

Answer:

It selects elements that are inside another specified element.

Next concepts that connect with this:

► Child Selector (>)

► Adjacent Sibling (+)

► General Sibling (~)

You're building real CSS power now 💪

     ...

sibling selector

Great — **Sibling Selectors** are extremely important for layouts & UI behavior 🔥

There are **two types of sibling selectors** in CSS:

1. Adjacent Sibling Selector (+)

Selects the **very next** sibling element.

Syntax:

CSS

```
element1 + element2 {  
    property: value;  
}
```

Example

 Copy code

html

```
<h2>Title</h2>  
<p>First paragraph</p>  
<p>Second paragraph</p>
```

CSS

```
h2 + p {  
    color: red;  
}
```

Only First paragraph becomes red.

 Copy code

2. General Sibling Selector (~)

Selects **all** matching siblings after the first element.

Syntax:

CSS

```
element1 ~ element2 {  
    property: value;  
}
```

Example

 Copy code

CSS

```
h2 ~ p {  
  color: blue;  
}
```

Both paragraphs become blue.

 Copy code

Key Difference

Selector	Affects
+	Only the immediate next element
~	All following siblings

Real UI Example

html

```
<label>Email</label>  
<input type="email">  
<small>Error message</small>
```

CSS

```
input:focus + small {  
  color: red;  
}
```

Shows error message only when input is focused.

 Copy code

Another Example

CSS

```
.menu h3 ~ p {  
  color: gray;  
}
```

 Copy code

Styles all `<p>` after `<h3>` inside `.menu` .

Specificity

Sibling selectors are **stronger than element selectors** but weaker than class & ID.

Interview Question

Q: Difference between `+` and `~` ?

Answer:

- `+` selects the immediate sibling,
 - `~` selects all following siblings.
-

Practice Challenge

html

```
<h1>Heading</h1>  
<p>Para 1</p>  
<p>Para 2</p>
```

Make only Para 1 red:

 Copy code

css

```
h1 + p { color: red; }
```

 Copy code

Next powerful concept:

► Attribute Selectors

(Used heavily in forms & frameworks)



child selector

Excellent — **Child Selector** is one of the most important layout selectors in CSS 💪
It gives you **precise control** over elements.

Child Selector (>)

Selects only the **direct children** of an element.

Syntax

CSS

```
parent > child {  
  property: value;  
}
```

 Copy code

Basic Example

HTML

html

```
<div>  
  <p>One</p>  
  <section>  
    <p>Two</p>  
  </section>  
</div>
```

CSS

CSS

```
div > p {  
  color: red;  
}
```

Result

 Copy code

- **One** → red
- **Two** → unchanged

Because only the **direct** `<p>` **child** is selected.

Child vs Descendant

Selector	Selects
<code>div p</code>	All <code><p></code> inside <code><div></code>
<code>div > p</code>	Only direct child

Real Website Example

CSS

```
nav > a {  
  padding: 10px;  
}
```

Only styles links that are directly inside `<nav>` .

 Copy code

Another Example

html

```
<ul>
  <li>Home</li>
  <li>
    About
    <ul>
      <li>Team</li>
    </ul>
  </li>
</ul>
```

CSS

```
ul > li {
  color: green;
}
```

Only first-level items become green.

 Copy code

Specificity

More specific than descendant selector.

Interview Question

Q: Why use child selector instead of descendant selector?

Answer:

To avoid applying styles to deeply nested elements and maintain strict layout control.

Mini Practice

html

```
<div>
  <span>A</span>
  <p>B</p>
```

```
<section>
  <span>C</span>
</section>
</div>
```

Make only A & B blue:

CSS

```
div > span, div > p {
  color: blue;
}
```

 Copy code

Next topic to master:

► Attribute Selector

(super useful in forms, login pages, validations)



attribute selector

Perfect — **Attribute Selector** is extremely powerful in real projects & forms 🔥

Let's master it clearly.

Attribute Selector

It selects elements based on the **presence or value of attributes**.

1. Select by Attribute Presence

CSS

```
input[type] {  
    border: 2px solid blue;  
}
```

[Copy code](#)

Selects all inputs that have a `type` attribute.

2. Select by Exact Value

CSS

```
input[type="text"] {  
    background: lightyellow;  
}
```

Only text inputs.

[Copy code](#)

3. Starts With ^

CSS

```
a[href^="https"] {  
    color: green;  
}
```

Links starting with `https` .

[Copy code](#)

4. Ends With \$

CSS

```
img[src$=".png"] {  
    border: 2px solid red;  
}
```

Only PNG images.

[Copy code](#)



5. Contains *

CSS

```
a[href*="google"] {  
  color: orange;  
}
```

Any link containing "google".

Copy code



6. Space-separated Value ~

CSS

```
div[class~="box"] {  
  background: lightblue;  
}
```

Matches one whole word in class list.

Copy code



7. Hyphen-separated |

CSS

```
p[lang|="en"] {  
  color: blue;  
}
```

Matches en or en-US .

Copy code



Real Login Form Example

CSS

```
input[type="password"] {  
  border: 2px solid red;  
}
```

```
input[required] {  
  background: #f2f2f2;  
}
```

Specificity

[Copy code](#)

Same level as class selector.

Interview Question

Q: Why are attribute selectors important?

Answer:

They allow styling without adding extra classes and provide dynamic control over elements.

Mini Practice

html

```
<input type="text">  
<input type="email">  
<input type="password">
```

Make only email input green:

[Copy code](#)

css

```
input[type="email"] {  
  background: lightgreen;  
}
```

[Copy code](#)

Next powerful topics:

► Pseudo-classes

► Pseudo-elements

► CSS Specificity & Cascade



pseudo class

Excellent — **Pseudo-classes** are what make websites feel **interactive & alive** ⚡

This is a very important topic.

Pseudo-Class

A **pseudo-class** selects an element based on its **state**.

Syntax

CSS

```
selector:pseudo-class {  
    property: value;  
}
```

 Copy code

Most Important Pseudo-Classes

 :hover

When mouse is over element

CSS

```
button:hover {  
    background: green;
```

```
color: white;  
}
```

 Copy code**:active**

When element is clicked

CSS

```
button:active {  
  transform: scale(0.95);  
}
```

 Copy code**:focus**

When input is selected

CSS

```
input:focus {  
  border: 2px solid blue;  
}
```

 Copy code**:visited**

After link is opened

CSS

```
a:visited {  
  color: purple;  
}
```

 Copy code**:checked**

When checkbox/radio is selected

CSS

```
input:checked {  
  accent-color: green;  
}
```

 Copy code

:first-child

CSS

```
li:first-child {  
  color: red;  
}
```

 Copy code

:last-child

CSS

```
li:last-child {  
  color: blue;  
}
```

 Copy code

:nth-child()

CSS

```
li:nth-child(2) {  
  color: orange;  
}
```

 Copy code

UI Example (Login Form)

CSS

```
input:focus {  
    outline: none;  
    border-color: #0077ff;  
}  
  
button:hover {  
    background: #0055cc;  
}
```



Interview Question

Q: What is pseudo-class?

[Copy code](#)

Answer:

A pseudo-class selects elements based on their state or position.



Mini Practice

html

```
<a href="#">Link</a>
```

CSS

```
a:hover { color: red; }  
a:visited { color: green; }
```

[Copy code](#)[Copy code](#)

Next big concept:

► Pseudo-Elements

(::before , ::after , etc.)



cascading and specificity

Excellent topic 🙌

Cascading & Specificity decide **which CSS rule actually works** when many rules target the same element.

Let's make this crystal clear 🧠

Cascading in CSS

Cascading means:

When multiple CSS rules apply to the same element, CSS decides **which one wins**.

CSS follows this order:

1. Importance

CSS

```
p { color: blue; }  
p { color: red !important; } → wins
```

2. Specificity

 Copy code

More specific selector wins.

3. Source Order

Last written rule wins.

Specificity System

Each selector has a score:

Selector	Value
Inline style	1000
ID	100
Class, attribute, pseudo-class	10
Element, pseudo-element	1
Universal *	0

Example:

CSS

```
#title .box p → 100 + 10 + 1 = 111
```

Copy code

Who Wins?

CSS

```
p { color: blue; }           /* 1 */
.text { color: green; }      /* 10 */
#para { color: red; }        /* 100 */
```

For:

Copy code

html

```
<p id="para" class="text">Hello</p>
```

Copy code

Red wins

When Specificity is Same → Last Rule Wins

CSS

```
p { color: blue; }  
p { color: red; }
```

 Copy code

👉 Red wins.

🚫 Avoid This Mistake

CSS

```
#title { color: red !important; } ❌ bad practice
```

 Copy code

Use `!important` only as last option.

🧪 Real Case

CSS

```
.nav a { color: blue; } /* 11 */  
header nav a { color: red; } /* 3 */
```

`.nav a` wins because $11 > 3$

 Copy code

🧠 Interview Question

Q: Difference between cascading and specificity?

Answer:

Cascading decides which rule applies; specificity is the scoring system that helps decide that.

🧪 Quick Practice

Who wins?

CSS

```
div p { color: blue; }    /* 2 */  
p.text { color: green; } /* 11 */
```

👉 Green wins

📋 Copy code

Now you officially understand the **core of CSS** 🔥

If you want, next I can teach you:

- ➡ Box Model
- ➡ Display & Position
- ➡ Flexbox
- ➡ Grid

📋 👍 💬 ⬆️ ↺ ...

important in css

Great — `!important` is **small but extremely powerful** in CSS ⚠️

And also **dangerous** if misused.

Let's understand it properly.

!important in CSS

`!important` forces a rule to **override everything else**, even high-specificity selectors.

🖋 Syntax

CSS

```
selector {  
  property: value !important;  
}
```

 Copy code

Example

CSS

```
p { color: blue; }  
#para { color: red; }  
  
p { color: green !important; }
```

For:

html

```
<p id="para">Hello</p>
```

 Result: **Green** Copy codeBecause **!important** beats ID.

Priority Order

1. **!important**
2. Inline styles
3. ID
4. Class / Attribute / Pseudo-class
5. Element
6. Universal

If **both** have **!important** , then **specificity** is used.

When both have **!important**

CSS

```
p { color: blue !important; }  
#para { color: red !important; }
```

 **Red** wins (ID has higher specificity). Copy code

When should you use it?

- ✓ Fix third-party library CSS
- ✓ Override browser default styles
- ✓ Temporary debugging

When NOT to use it

- ✗ Regular styling
- ✗ Layout design
- ✗ Large projects

Overusing it makes CSS **unmaintainable**.

Real Example

CSS

```
button {  
  background: gray;  
}  
  
.bootstrap-btn {  
  background: blue !important;  
}
```

Interview Question

 Copy code

Q: Why is `!important` discouraged?

Answer:

Because it breaks the natural CSS hierarchy and makes code hard to maintain.

Mini Practice

CSS

```
p { color: red; }  
.text { color: blue !important; }
```

For:

 Copy code

html

```
<p class="text">Test</p>
```

 Copy code Blue

You now know how CSS really decides everything 🗨️ 🔥

Next topics that make you pro:

-  Box Model
-  Display Property
-  Positioning
-  Flexbox & Grid

     ...

inheritance

Excellent — **Inheritance in CSS** is one of the core principles that makes CSS powerful and efficient 🗨️ 🔥

Let's understand it clearly.

Inheritance in CSS

Inheritance means:

Some CSS properties applied to a parent element are automatically passed down to its children.

Basic Example

CSS

```
div {  
  color: blue;  
}
```

html

```
<div>  
  <p>Hello</p>  
  <span>World</span>  
</div>
```

Both **Hello** and **World** become blue because `color` is **inherited**.

 Copy code

Properties That Inherit Naturally

Property	Inherits
color	✓
font-family	✓
font-size	✓
visibility	✓
cursor	✓
line-height	✓

Properties That Do NOT Inherit

Property	Inherits
margin	✗
padding	✗
border	✗
background	✗
width/height	✗

Force Inheritance

CSS

```
button {  
    color: inherit;  
}
```

Forces button text to inherit from parent.

 Copy code

Stop Inheritance

CSS

```
p {  
    color: initial;  
}
```

or

 Copy code

CSS

```
p {  
    color: unset;  
}
```

```
}
```

 Copy code

Real UI Example

CSS

```
body {  
  font-family: Arial;  
  color: #333;  
}
```

All text on the website inherits this automatically.

 Copy code

Inheritance vs Specificity

If a child has its own rule, it overrides inherited value.

CSS

```
div { color: red; }  
p { color: blue; }
```

p becomes blue.

 Copy code

Interview Question

Q: What is inheritance in CSS?

Answer:

It is the process by which some CSS properties are passed from parent to child elements.

Mini Practice

CSS

```
section { font-size: 20px; }
```

 Copy code

Everything inside `<section>` will automatically become 20px unless overridden.

You now understand one of the **core engines of CSS** 🌸

Next up (very important):

➡ **Box Model**



explain sibling

Sure, Aamir 😊

Let's explain **Sibling Selectors in CSS** very clearly and simply.

What are Sibling Selectors?

Sibling selectors select elements that **share the same parent**.

There are **two types**:

1. **Adjacent Sibling Selector (+)**
 2. **General Sibling Selector (~)**
-

1. Adjacent Sibling Selector (+)

Selects **only the very next sibling**.

Syntax

CSS

```
element1 + element2 {  
  property: value;  
}
```

 Copy code

Example

html

```
<h2>Title</h2>  
<p>First</p>  
<p>Second</p>
```

CSS

```
h2 + p {  
  color: red;  
}
```

Result

 Copy code

- First → red
- Second → unchanged

Because only the **first** `<p>` **immediately after** `<h2>` is selected.

2. General Sibling Selector (~)

Selects all matching siblings that come **after** the first element.

Syntax

CSS

```
element1 ~ element2 {  
  property: value;  
}
```

Example

 Copy code

CSS

```
h2 ~ p {  
  color: blue;  
}
```

Result

Both **First** and **Second** become blue.

 Copy code

Key Difference

Selector	Selects
+	Only the next element
~	All following elements

Real Website Example

html

```
<input>  
<small>Error message</small>
```

css

```
input:focus + small {  
  color: red;  
}
```

Error message becomes red **only when input is focused**.

 Copy code

Interview Question

Q: What is the difference between + and ~ ?

Answer:

- + selects the immediate next sibling,
- ~ selects all following siblings.

**Simple Practice**

html

```
<h1>Heading</h1>
<p>One</p>
<p>Two</p>
```

Make only **One** red:

Copy code

css

```
h1 + p { color: red; }
```

Copy code

If you'd like, next I can explain:

- Child vs Descendant vs Sibling selectors comparison
- Combinators in CSS (important interview topic)



sibling combinator

Great — now you're learning **real CSS architecture** 🧙Let's fully understand the **Sibling Combinators**.**Sibling Combinators in CSS****Combinators** define the relationship between selectors.

There are **two sibling combinators**:

1. **Adjacent Sibling Combinator (+)**
 2. **General Sibling Combinator (~)**
-

1 Adjacent Sibling Combinator (+)

Selects the **very next** sibling only.

Syntax

CSS

 Copy code

```
A + B {  
    property: value;  
}
```

Example

HTML

 Copy code

```
<h3>Heading</h3>  
<p>Para 1</p>  
<p>Para 2</p>
```

CSS

 Copy code

```
h3 + p {  
    color: red;  
}
```

Result:

Only **Para 1** becomes red.

2 General Sibling Combinator (~)

Selects **all matching siblings** after the first one.

Syntax

CSS

 Copy code

```
A ~ B {  
  property: value;  
}
```

Example

CSS

 Copy code

```
h3 ~ p {  
  color: blue;  
}
```

Result:

Para 1 and Para 2 both become blue.

Important Rules

- ✓ Both elements must share the **same parent**
- ✓ Only affects elements **after** the first element
- ✓ Does NOT select previous siblings

Comparison

Combinator	Effect
+	Only immediate next
~	All following

Real UI Example

html

 Copy code

```
<input>
<span>Error</span>
```

css

 Copy code

```
input:focus + span {
  display: block;
}
```

Shows error when input is focused.

Interview Question

Q: What is a sibling combinator?

Answer:

A sibling combinator selects elements that share the same parent and come after another element.

Mini Practice

html

 Copy code

```
<h1>Title</h1>
<p>A</p>
<p>B</p>
```

Make only B blue:

css

 Copy code

```
h1 + p + p { color: blue; }
```

If you'd like, next topic:

 [All CSS Combinators in one chart](#)

Combinators vs Selectors (interview focus)

